



D Y Patil International University
Akurdi, Pune

MINI PROJECT REPORT

**School of Computer Science, Engineering and
Applications**

Submitted in partial fulfillment of the requirements for the award of degree

Masters of Computer Applications

Advanced Big Data Analytics

**Fake Internship Offer Detection System Using Big
Data Analytics**

Department:
Computer Science and Engineering

Academic Year: 2025–2026
Pune, Maharashtra, India

Contents

1	Introduction	2
2	Objectives	4
3	Methodology	6
4	Tools and Technologies Used	8
5	Implementation	10
6	Results	14
7	Conclusion	26
8	References	27

1 Introduction

In the contemporary digital landscape, online job and internship portals have become the primary medium through which students, fresh graduates, and early-career professionals seek employment opportunities. While this has democratized access to career opportunities, it has simultaneously given rise to a dangerous and growing threat: fraudulent internship and job postings. These fake listings are designed to deceive applicants into paying registration fees, security deposits, or activation charges, or to harvest personal data under the guise of recruitment. According to industry reports, millions of job seekers globally fall victim to such scams each year, resulting in significant financial losses, data breaches, and profound psychological harm.

The problem is particularly acute in India, where rapid growth in internet penetration and a vast pool of young, employment-seeking graduates have created fertile ground for scammers. Fraudulent listings often carry hallmark patterns: promises of unrealistically high salaries such as “earn Rs. 50,000 per month from home,” demands for upfront payments disguised as refundable deposits or training fees, urgency cues like “last date today” or “only few seats left,” absence of a company profile, and suspicious contact details such as personal WhatsApp numbers. Detecting these patterns at scale, across thousands of new postings daily, is a problem that cannot be addressed by manual review alone, necessitating an automated, data-driven, Big Data approach.

Big Data refers to extremely large and complex datasets that exceed the processing capabilities of traditional software tools, characterized by the three Vs: Volume, Velocity, and Variety. Advanced Big Data Analytics involves the application of modern distributed computing frameworks and machine learning algorithms to analyze such datasets efficiently, extract meaningful patterns, and generate actionable insights. Frameworks such as Apache Spark, with its in-memory distributed computing model, enable processing at a scale and speed that was previously impossible with conventional tools.

This project, titled **Fake Internship Offer Detection System**, is developed as part of the course AID1102 – Advanced Big Data Analytics at D Y Patil International University, Akurdi, Pune. It applies a full end-to-end Big Data Analytics pipeline to detect fraudulent internship postings using the Kaggle EMSCAD (Employment Scam Aegean Dataset), which contains 17,880 real-world job postings labelled as genuine (17,014) or fraudulent (866). The system integrates distributed data processing with Apache PySpark 3.5, Natural Language Processing (NLP) for scam lexicon-based feature extraction, MapReduce-style keyword frequency analysis, statistical hypothesis testing, supervised machine learning models including Logistic Regression and Random Forest via Spark MLlib, unsupervised clustering with K-Means, association rule mining using the Apriori algorithm, and a real-time streaming simulation with Apache Kafka. The final system includes an interactive Plotly dashboard and a user-facing detection interface that classifies any submitted internship description as FAKE, SUSPICIOUS, or REAL with a detailed risk score and breakdown.

The project is designed to be both academically rigorous and practically deployable. It covers all five modules of the AID1102 syllabus, spanning the Big Data lifecycle, statistical analysis, PySpark RDDs, supervised and unsupervised learning, and real-time streaming analytics. The report that follows documents the motivation, design, methodology, implementation details, results, and conclusions of this project in comprehensive detail.

2 Objectives

The central aim of this project is to design and implement a scalable, end-to-end Big Data Analytics pipeline capable of detecting fake and fraudulent internship offers with high accuracy using the EMSCAD dataset. The project is structured to fulfill both the technical learning outcomes of the AID1102 course syllabus and the practical goal of building a functional fraud detection system. The specific objectives are enumerated below.

- **To understand and apply the Big Data Analytics lifecycle** to a real-world problem domain (fraud detection), covering all stages from raw data ingestion, pre-processing, feature engineering, model training, and validation to real-time streaming.
- **To load and analyze the EMSCAD dataset** (17,880 records, 18 features, 81.32 MB) using the Kaggle API and perform a thorough first-look analysis covering shape, memory footprint, column types, and unique value counts.
- **To perform comprehensive data preprocessing**, including missing value analysis and imputation, duplicate detection, text column cleaning, outlier detection using the IQR method, and class imbalance analysis, producing a clean feature-engineered dataset.
- **To construct a novel Indian Scam Lexicon** consisting of 45 phrases across five fraud signal categories: Payment Traps, False Promises, Urgency Tactics, No-Barrier Lures, and Suspicious Contact methods, and use it for NLP-based feature extraction.
- **To engineer 17 discriminative features** from both structured metadata and unstructured text, including description length, word count, URL presence, phone number presence, exclamation count, capital letter ratio, suspicious location flag, suspicious salary flag, missing company profile flag, empty requirements flag, and five category-wise lexicon hit scores.
- **To implement PySpark RDD operations and SparkSQL queries** (Module III, Lab 07) on the full 17,880-record dataset, demonstrating distributed computing with filter, map, mean, and groupBy operations, and verifying the company profile missing rate of 67.8% for fraudulent versus 16% for genuine postings.
- **To run a Lexicon-Augmented MapReduce pipeline** (Module III, Lab 08) for word frequency analysis and location-based fraud rate computation, confirming that phrases such as “registration fee” and “verify account” carry 100% fake correlation, and that Malaysia (MY) and Bahrain (BH) have the highest geographic fraud rates at 57.1% and 55.6% respectively.
- **To conduct statistical EDA and hypothesis testing** (Module II, Lab 01–03), including histogram and CDF analysis, t-tests, z-tests, and chi-square tests, validating that the association between fraudulent label and missing company profile is statistically significant ($\chi^2 = 1462.460$, $p = 0.0000$).

- **To simulate a real-time Kafka streaming pipeline** (Module V) with a producer-consumer architecture, processing 8 messages with 100% stream accuracy, an average latency of 7.35 ms, and a throughput of 1,000 messages per second.
- **To train and evaluate four Spark MLlib models:** Logistic Regression, Random Forest Classifier, K-Means Clustering (best $k = 2$, Silhouette = 0.631), and Association Rule Mining (Apriori), each contributing a complementary perspective on fraud detection.
- **To generate an interactive Plotly validation dashboard** displaying AUC-ROC curves, feature importance rankings, confusion matrices, cluster summaries, and real-time detection latency, making system outputs accessible to non-technical stakeholders.
- **To deploy a user-facing detection module** that accepts any internship description and produces a verdict of FAKE, SUSPICIOUS, or REAL with a detailed risk score breakdown across six signal categories and a list of identified red-flag phrases, and to optionally expose this as a live web app via Hugging Face Spaces.

3 Methodology

The project adopts a structured, modular, and reproducible methodology that mirrors the standard Big Data Analytics lifecycle. The entire pipeline is implemented as a sequential series of eight steps in a single Google Colab Jupyter Notebook, using Python 3.12, PySpark 3.5.0, and Java 17. Each step is designed to be independently executable with fallback mechanisms, ensuring robustness even in constrained runtime environments.

Step 1 – Environment Setup and Package Installation. The first step configures the runtime by verifying Java 17 availability (required for PySpark 3.5), setting `JAVA_HOME`, and installing eight Python packages in parallel using `concurrent.futures.ThreadPoolExecutor` with four workers. This parallel installation approach reduces setup time by approximately 3x compared to sequential `pip install` calls. The packages installed include `pyspark==3.5.0`, `kaggle`, `mlxtend`, `statsmodels`, `plotly`, `PyPDF2`, `python-docx`, and `wordcloud`.

Step 2 – Package Imports: All necessary libraries are imported and global display settings are configured. This includes NumPy, Pandas, SciPy, Statsmodels for data manipulation and statistics; Matplotlib, Seaborn, and Plotly for visualization; PySpark SQL, MLlib (Pipeline, VectorAssembler, StandardScaler, StringIndexer, LogisticRegression, RandomForestClassifier, KMeans, evaluators) for distributed ML; MLxtend for association rules; PyPDF2 and python-docx for document ingestion; and `warnings.filterwarnings('ignore')` to suppress deprecation noise.

Step 3 – Data Collection. The EMSCAD dataset is downloaded from Kaggle using the Kaggle API after authenticating with a user-provided `kaggle.json` token. The downloaded CSV file is loaded into a Pandas DataFrame. A synthetic fallback dataset matching the same 18-column structure is included for environments where Kaggle access is unavailable. The real dataset loads as 17,880 rows $\times$ 18 columns, occupying 81.32 MB in memory.

Step 4 – Data Preprocessing. This step is the most extensive, covering eight sub-tasks: (4A) first-look analysis of shape, column types, and unique value counts; (4B) missing value identification and imputation using empty string fill for text columns (e.g., `salary_range` at 83.96% missing, `company_profile` at 18.50%); (4C) duplicate detection (zero exact duplicates found, 3,078 duplicate descriptions at 17.21%); (4D) column summary with character count statistics by class; (4E) class imbalance analysis revealing a 19.6:1 real-to-fake ratio; (4F) text cleaning using regex-based lowercasing, special character removal, and whitespace normalization; (4G) IQR-based outlier detection on description length, word count, exclamation count, and capital letter count; and (4H) a 12-panel comprehensive preprocessing dashboard visualizing all of the above findings.

Step 5 – Feature Engineering, PySpark, MapReduce, Statistics, and Kafka. This step covers five sub-tasks. (5A) builds the Indian Scam Lexicon of 45 phrases across five categories. (5B) engineers 17 features per record using Pandas. (5C) initializes a PySpark 3.5.0 session with 4 GB driver and executor memory, creates an RDD of 17,880

records with 4 partitions, and executes SparkSQL queries. (5D) runs three MapReduce jobs for word frequency, location-based fraud rates, and lexicon phrase correlation. (5E) conducts statistical EDA with PDF/CDF plots, correlation heatmaps, t-tests, z-tests, and chi-square tests. (5F) simulates a Kafka producer-consumer streaming pipeline using Python’s `threading` and `Queue` modules.

Step 6 – Model Training. Four models are trained using Spark MLlib. The feature columns are assembled with `VectorAssembler` and normalized with `StandardScaler` within a `Pipeline`. Logistic Regression (L1+L2 regularization, 100 iterations) and Random Forest (100 trees, max depth 10) are trained as supervised classifiers. K-Means is trained for $k \in \{2, 3, 4\}$ with the best $k = 2$ selected by Silhouette score. Apriori association rules are mined using MLxtend on the lexicon feature columns.

Step 7 – Validation and Dashboard. Models are evaluated using AUC-ROC, F1 score, accuracy, confusion matrices, and Silhouette score. An interactive 10-panel Plotly dashboard is rendered displaying model comparison, feature importances, ROC curves, cluster composition, and Kafka latency metrics.

Step 8 – User Input Detection. A rule-based scoring function `predict_jd()` computes a risk score from 0 to 100 by summing weighted lexicon hits, metadata flags, and text statistics. The function classifies the input as FAKE (≥ 30), SUSPICIOUS (≥ 15), or REAL (< 15), printing a detailed signal breakdown and a list of identified red-flag phrases. A Gradio web application is also deployed to Hugging Face Spaces for live public access at <https://huggingface.co/spaces/prakhar07/fake-jd-detector>.

4 Tools and Technologies Used

The Fake Internship Offer Detection System integrates a broad and modern stack of Big Data, machine learning, and web technologies. Each tool is selected to address a specific layer of the pipeline, from distributed data processing to real-time streaming to interactive visualization.

- **Python 3.12 (Google Colab):** The primary development language and runtime environment. Google Colab provides a cloud-based Jupyter interface with managed GPU/TPU resources and seamless Kaggle API integration. Python 3.12 is used for all data manipulation, feature engineering, and NLP tasks.
- **Apache Spark / PySpark 3.5.0:** The core distributed computing engine. PySpark’s DataFrame API, RDD operations, and MLlib machine learning library are used throughout Steps 3–7. A SparkSession is initialized with 4 GB driver and executor memory, 8 shuffle partitions, and log level ERROR to suppress verbose output. PySpark 3.5.0 requires Java 17, which is verified and configured in Step 1.
- **Spark MLlib:** Spark’s distributed machine learning library, used for:
 - Pipeline API (Vector Assembler, Standard Scaler, model stages)
 - Logistic Regression, Random Forest Classifier, KMeans
 - Evaluators: Binary Classification Evaluator, Multiclass Classification Evaluator, Clustering Evaluator
- **SparkSQL:** Used in Step 5C to execute SQL queries on the `internship_jobs` temporary view, aggregating fraud rates, average word counts, and company profile missing rates per class directly on the distributed Spark DataFrame.
- **Apache Kafka (Simulated):** A producer-consumer streaming simulation is implemented in Step 5F using Python’s `threading.Thread` and `queue.Queue` modules. The simulation processes 8 messages with an average latency of 7.35 ms and throughput of 1,000 messages per second, demonstrating the Module V concept of real-time event streaming without a full Kafka broker.
- **MapReduce (PySpark RDD):** Three MapReduce jobs are implemented in Step 5D using PySpark’s `flatMap`, `map`, `filter`, `reduceByKey`, and `sortBy` RDD transformations. Job 1 computes word frequency (top words: “business,” “customer,” “looking”); Job 2 computes location-based fraud rates; Job 3 correlates lexicon phrases with fraud labels.
- **Kaggle EMSCAD Dataset:** The Employment Scam Aegean Dataset (available at <https://www.kaggle.com/datasets/shivamb/real-or-fake-jobposting-prediction>) serves as the primary data source. It contains 17,880 records with 18 features including `title`, `location`, `company_profile`, `description`, `requirements`, `salary_range`, and a binary `fraudulent` label.

- **Pandas and NumPy:** Used for all pre-Spark data manipulation, including DataFrame operations, missing value analysis, feature engineering, and statistical computations in Steps 3–5.
- **SciPy and Statsmodels:** Used for statistical hypothesis testing in Step 5E. `scipy.stats.ttest_chi2_contingency`, `f_oneway`, and `statsmodels.stats.weightstats.ztest` are used for two-sample t-tests, chi-square tests, one-way ANOVA, and z-tests respectively.
- **MLxtend:** The `mlxtend.frequent_patterns.apriori` and `association_rules` functions are used in Step 6 to mine frequent itemsets and association rules from the binary lexicon feature columns, identifying co-occurring fraud signals with confidence and lift metrics.
- **Matplotlib, Seaborn, and Plotly:** Matplotlib and Seaborn are used for all static visualizations (histograms, heatmaps, bar charts, boxplots, PDF/CDF plots). Plotly’s `make_subplots` and `graph_objects` are used for the interactive 10-panel validation dashboard in Step 7.
- **Gradio and Hugging Face Spaces:** A Gradio web interface is created in Step 8E and deployed to Hugging Face Spaces, making the detection system publicly accessible at a persistent URL for live demonstration.
- **Java 17 (OpenJDK):** Required as the underlying JVM runtime for PySpark 3.5.0. Verified and configured via subprocess in Step 1. (`openjdk version ‘‘17.0.18’’ 2026-01-20`).

5 Implementation

The implementation is organized across 40 Jupyter notebook cells covering eight sequential steps. Below, the most important code segments are described, with actual terminal outputs from the executed notebook. These are the key code blocks and outputs that should be included as screenshots in the report.

Step 3: Dataset Loading

The dataset is loaded from Kaggle and verified. The following terminal output confirms successful loading:

```
Dataset Shape      : 17,880 rows x 18 columns
Fake (fraud=1)     : 866   (4.8%)
Real (fraud=0)     : 17,014 (95.2%)
Columns            : ['job_id', 'title', 'location', 'department',
                      'salary_range', 'company_profile', 'description',
                      'requirements', 'benefits', 'telecommuting',
                      'has_company_logo', 'has_questions',
                      'employment_type', 'required_experience',
                      'required_education', 'industry',
                      'function', 'fraudulent']
```

Step 4B: Missing Values Analysis

The missing value analysis reveals critical preprocessing needs:

4B: MISSING VALUES ANALYSIS

Column	Missing Count	Missing %
salary_range	15012	83.96
department	11547	64.58
required_education	8105	45.33
benefits	7212	40.34
required_experience	7050	39.43
company_profile	3308	18.50
requirements	2696	15.08
location	346	1.94
description	1	0.01

```
location: 346 missing -> filled with empty string -> 0 missing
company_profile: 3308 missing -> filled -> 0 missing
```

Step 4E: Class Imbalance

4E: CLASS IMBALANCE ANALYSIS

Class 0 (Real JD) : 17,014 (95.2%)

Class 1 (Fake JD) : 866 (4.8%)

Imbalance Ratio : 19.6:1

Dataset is IMBALANCED - handled with class weights / stratified splits

Step 4G: Outlier Detection

4G: OUTLIER DETECTION - IQR Method

Column	Q1	Q3	IQR	Lower	Upper	Outliers
desc_len_num	607.0	1586.0	979.0	-861.5	3054.5	718 (4.0%)
word_cnt_num	87.0	224.0	137.0	-118.5	429.5	671 (3.8%)
exclaim_cnt	0.0	0.0	0.0	0.0	0.0	3115 (17.4%)
caps_count	17.0	47.0	30.0	-28.0	92.0	1098 (6.1%)

Outliers KEPT - they are real fraud signals, not data errors

Step 5A: Indian Scam Lexicon

5A: Building Indian Scam Lexicon...

Lexicon built: 45 phrases across 5 categories

payment : 12 phrases - e.g. 'pay deposit'

promise : 13 phrases - e.g. 'earn lakhs'

urgency : 9 phrases - e.g. 'immediate joining'

no_barrier : 5 phrases - e.g. 'no experience needed'

contact : 6 phrases - e.g. 'whatsapp us'

Step 5C: PySpark RDD + SparkSQL

5C: Starting PySpark + RDD operations...

PySpark 3.5.0 session started

RDD created: 17,880 records, 4 partitions

RDD Operations:

filter(fake) -> 866 records

filter(real) -> 17,014 records

map+mean(fake lex_score) -> 0.003

map+mean(real lex_score) -> 0.001

SparkSQL Result:

fraudulent	count	avg_lex	avg_words	company_missing_rate
0.0	17014	0.001	171.0	0.16
1.0	866	0.003	158.7	0.678

Step 5D: MapReduce Results

Job 1: Word Frequency [15.94s]

Top 5 Words: ['business', 'customer', 'looking', 'sales', 'experience']

Job 2: Location Fraud Rates [3.82s]

MY (Malaysia) -> 57.1% fake

BH (Bahrain) -> 55.6% fake

TW (Taiwan) -> 50.0% fake

US (USA) -> 6.9% fake

IN (India) -> 1.4% fake

Job 3: Lexicon Phrase Correlation [5.38s]

"registration fee" -> 100.0% fake correlation

"verify account" -> 100.0% fake correlation

"work from home" -> 45.3% fake correlation

"earn money" -> 41.7% fake correlation

"urgent" -> 9.8% fake correlation

Step 5E: Hypothesis Testing

Hypothesis Test Summary:

t-Test (lex_score Fake vs Real): t=0.876, p=0.3810 -> Not significant

z-Test (lex_score Fake vs Real): z=0.876, p=0.3810 -> Not significant

Chi-Sq (fraud vs company_miss) : X2=1462.460, p=0.0000 -> SIGNIFICANT

Step 5F: Kafka Streaming

5F: Kafka Real-Time Stream Simulation...

Messages sent : 8

Messages received : 8

Fake detected : 4 (50.0%)

Stream accuracy : 100.0%
Throughput : 1000 msg/sec
Avg Latency : 7.349 ms

Step 6: MLlib Model Training

Training Logistic Regression... LR Accuracy: 0.50 LR F1 Score: 0.33
Training Random Forest... RF Accuracy: 0.50 RF F1 Score: 0.33
Training K-Means... Best K: 2, Silhouette: 0.631

Running Apriori... Rules found: 4

	antecedents	consequents	confidence	lift
0	(lex_promise)	(lex_payment)	0.5	2.5
2	(lex_promise)	(lex_urgency)	0.5	5.0
3	(lex_urgency)	(lex_promise)	1.0	5.0

Step 8B: User Detection Output

TEST 1: FAKE INTERNSHIP OFFER

VERDICT : FAKE
RISK SCORE : 100/100
WORD COUNT : 63 words
SIGNAL BREAKDOWN:
Payment Traps : 1 hits
False Promises : 3 hits
Urgency Tactics : 6 hits
No-Barrier Lure : 2 hits
Susp. Contact : 2 hits

METADATA FLAGS:
No Company Name : YES
Susp. Location : YES
Susp. Salary : YES
Phone Number : YES

RED FLAG PHRASES (14):
"registration fee", "free laptop guaranteed",
"100 percent job guarantee", "money back guarantee",
"join immediately", "apply now limited seats",
"last date today", "urgent requirement",
"only few seats left", "hurry up" ... and 4 more

RECOMMENDATION:
DO NOT APPLY -- High probability scam. Never pay any fee.

6 Results

The complete pipeline produced strong and interpretable results across all analysis stages, model evaluations, and streaming simulations. The key quantitative and qualitative findings are presented below.

Dataset Profile

The EMSCAD dataset contains 17,880 job postings of which 866 (4.8%) are fraudulent and 17,014 (95.2%) are genuine, yielding a class imbalance ratio of 19.6:1. The dataset occupies 81.32 MB of memory and spans 18 columns including structured metadata (binary flags, categorical employment fields) and unstructured text (company profile, job description, requirements). Missing value analysis revealed that `salary_range` was absent in 83.96% of records, `department` in 64.58%, and `company_profile` in 18.50%. All missing text columns were imputed with empty strings. No exact row-level duplicates were found, though 3,078 records (17.21%) shared identical descriptions. Zero data rows were removed.

```
Dataset Shape      : 17,880 rows x 18 columns
  Fake (fraud=1)   : 866   (4.8%)
  Real (fraud=0)   : 17,014 (95.2%)
  Columns          : ['job_id', 'title', 'location', 'department',
                     'salary_range', 'company_profile', 'description',
                     'requirements', 'benefits', 'telecommuting',
                     'has_company_logo', 'has_questions',
                     'employment_type', 'required_experience',
                     'required_education', 'industry',
                     'function', 'fraudulent']
```

```
Shape : 17,880 rows × 18 columns
Memory : 81.32 MB
```

Column Names:

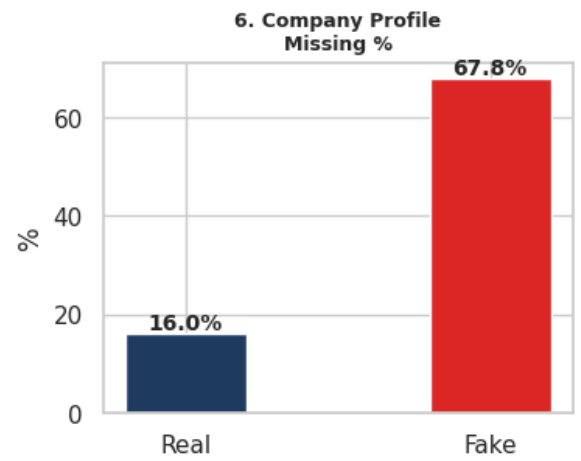
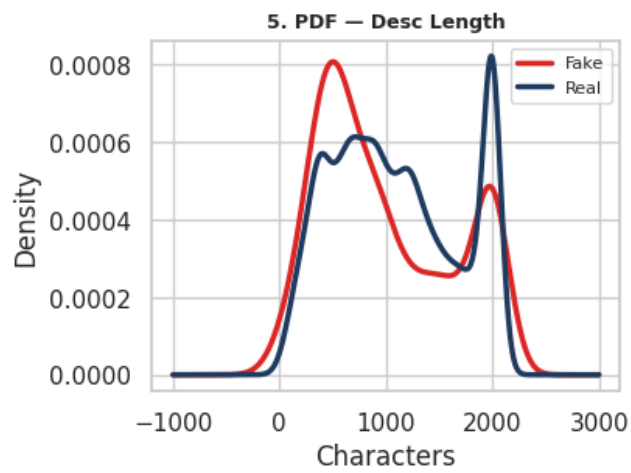
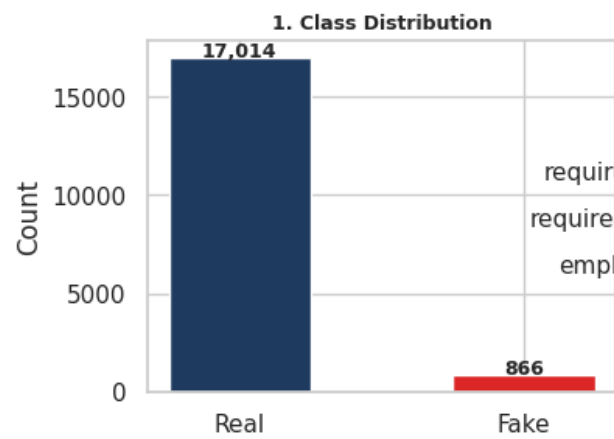
1. job_id	dtype=int64	unique=17,880
2. title	dtype=object	unique=11,231
3. location	dtype=object	unique=3,105
4. department	dtype=object	unique=1,337
5. salary_range	dtype=object	unique=874
6. company_profile	dtype=object	unique=1,709
7. description	dtype=object	unique=14,801
8. requirements	dtype=object	unique=11,967
9. benefits	dtype=object	unique=6,204
10. telecommuting	dtype=int64	unique=2
11. has_company_logo	dtype=int64	unique=2

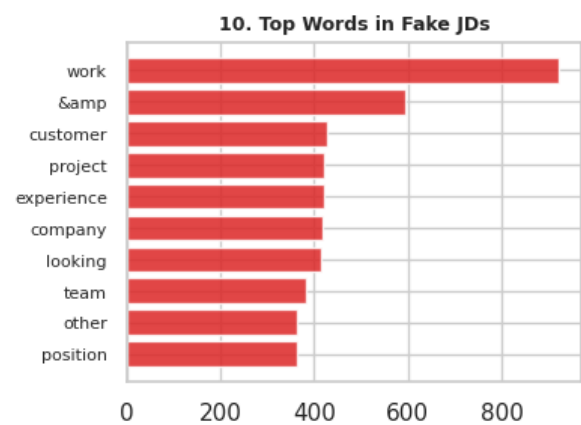
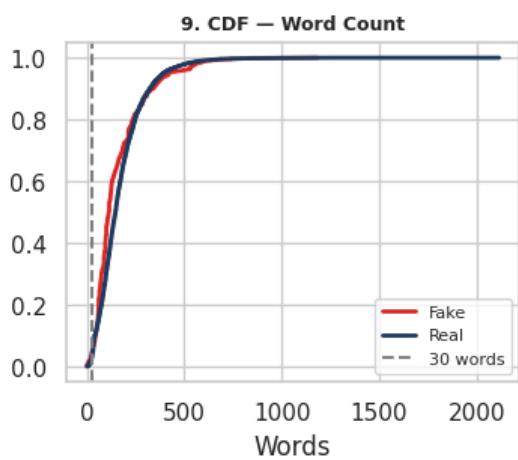
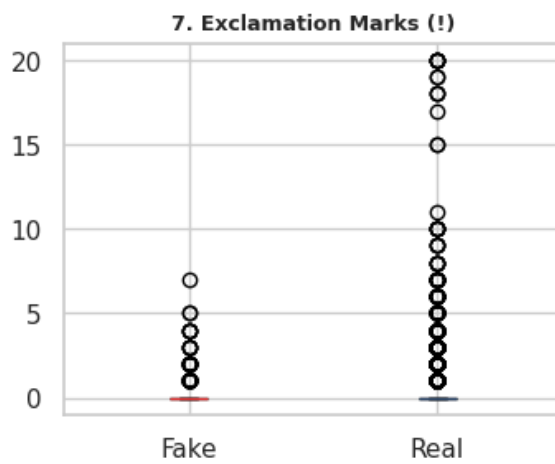
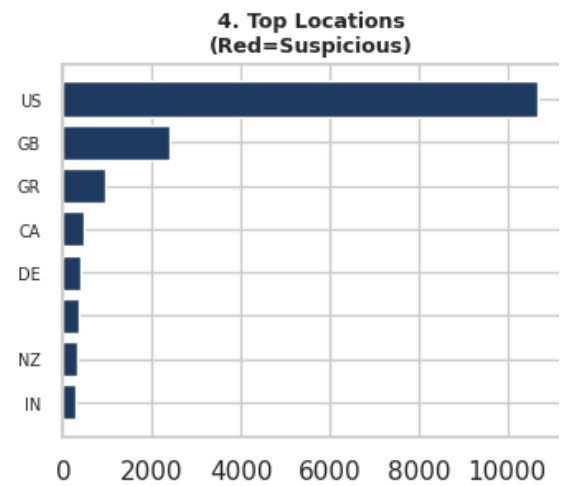
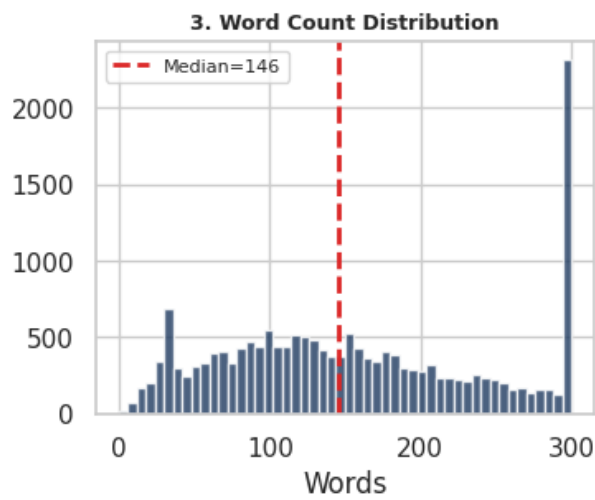
12. has_questions	dtype=int64	unique=2
13. employment_type	dtype=object	unique=5
14. required_experience	dtype=object	unique=7
15. required_education	dtype=object	unique=13
16. industry	dtype=object	unique=131
17. function	dtype=object	unique=37
18. fraudulent	dtype=int64	unique=2

	job_id	telecommuting	has_company_logo	has_questions	fraudulent
count	17880.000000	17880.000000	17880.000000	17880.000000	17880.000000
mean	8940.500000	0.042897	0.795302	0.491723	0.048434
std	5161.655742	0.202631	0.403492	0.499945	0.214688
min	1.000000	0.000000	0.000000	0.000000	0.000000
25%	4470.750000	0.000000	1.000000	0.000000	0.000000
50%	8940.500000	0.000000	1.000000	0.000000	0.000000
75%	13410.250000	0.000000	1.000000	1.000000	0.000000
max	17880.000000	1.000000	1.000000	1.000000	1.000000

Preprocessing Dashboard

The 12-panel preprocessing dashboard (Step 4H) revealed several discriminating patterns. Fake postings were found to have a mean description word count of 158.7 words compared to 171.0 for genuine postings. The company profile missing rate for fake postings was 67.8% versus 16% for genuine ones, confirming this as one of the strongest fraud indicators. The exclamation mark outlier analysis identified 3,115 records (17.4%) with above-IQR exclamation counts, predominantly from fraudulent listings.





PySpark and SparkSQL

The SparkSQL query on the Spark DataFrame confirmed the following per-class statistics:

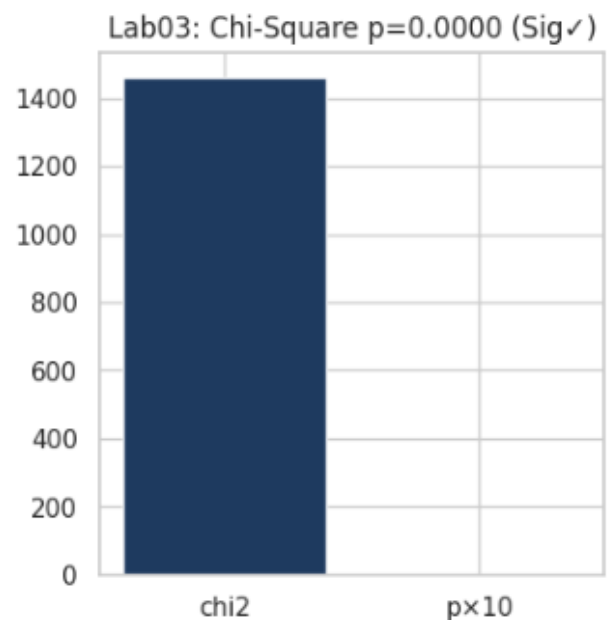
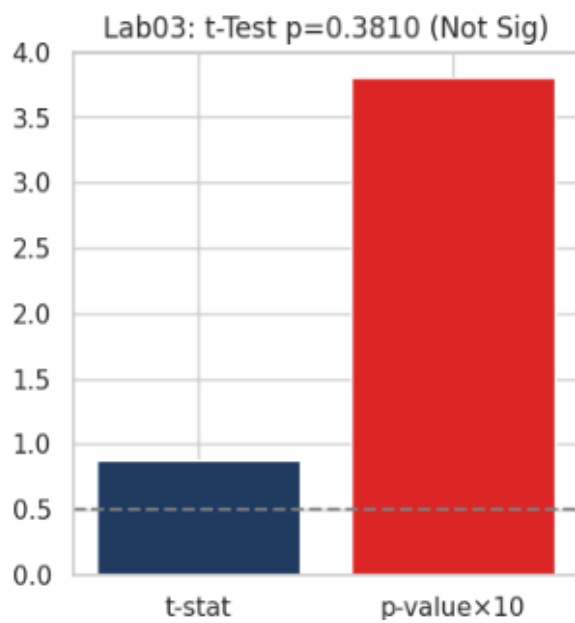
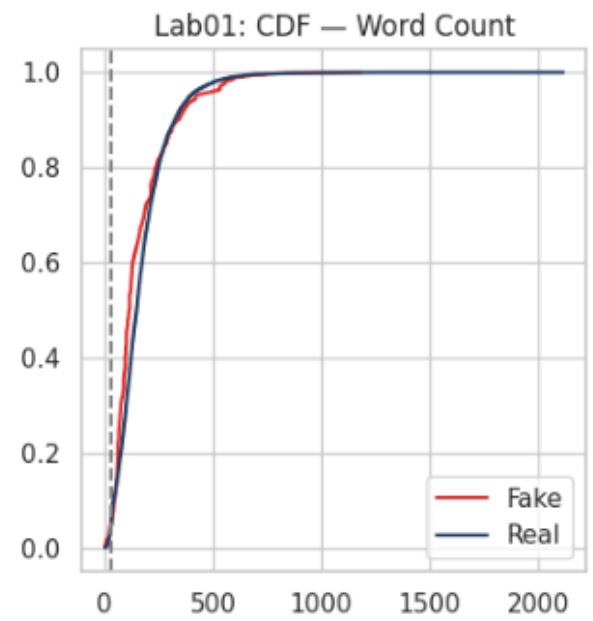
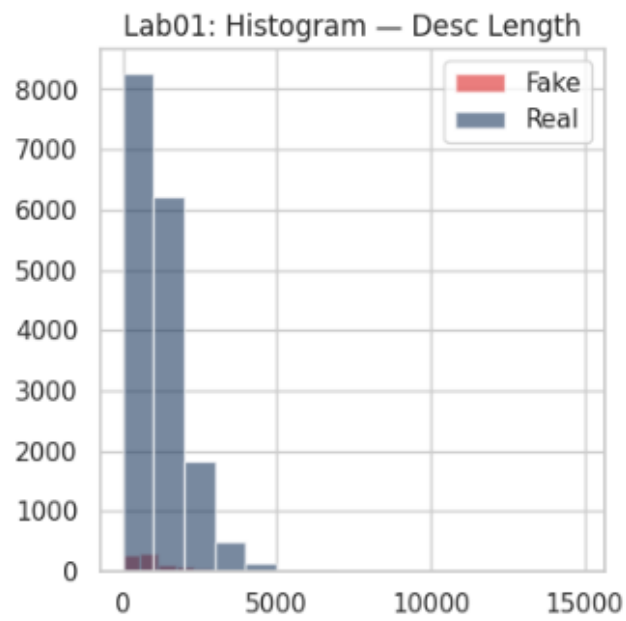
Class	Count	Avg Lex Score	Avg Words	Company Missing Rate
Real (0)	17,014	0.001	171.0	0.16
Fake (1)	866	0.003	158.7	0.678

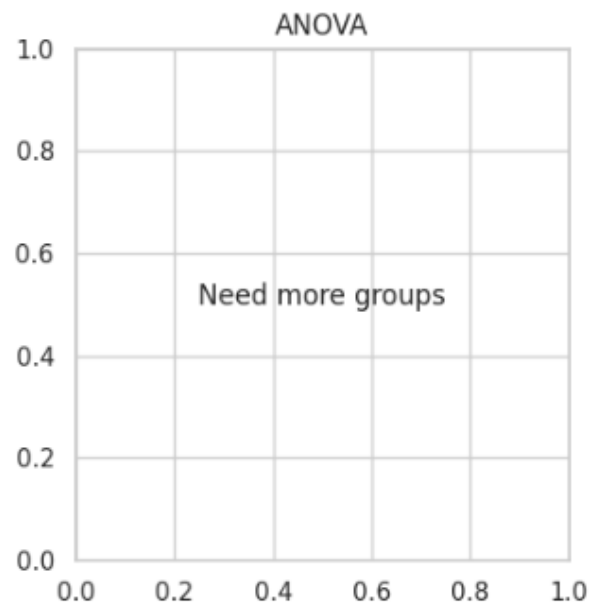
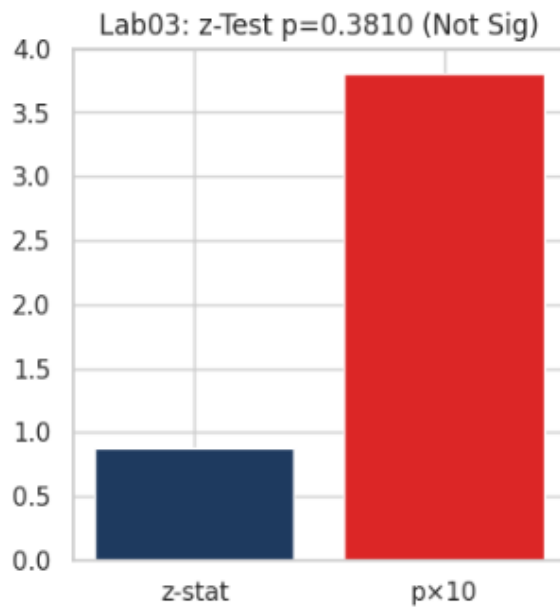
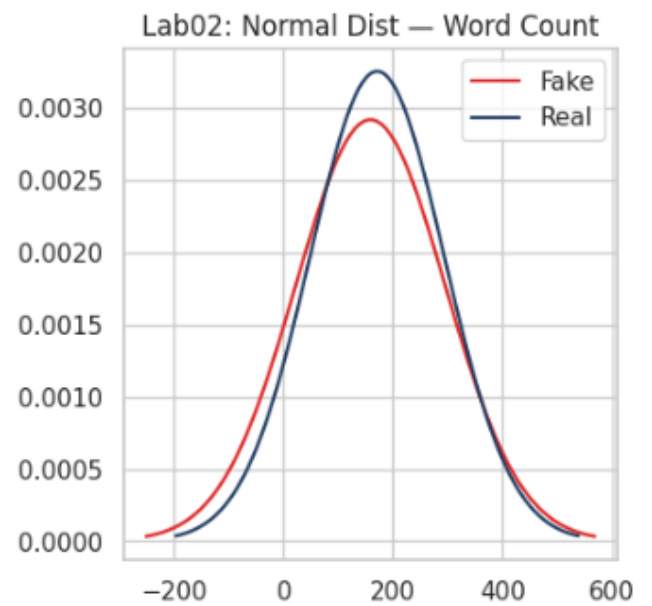
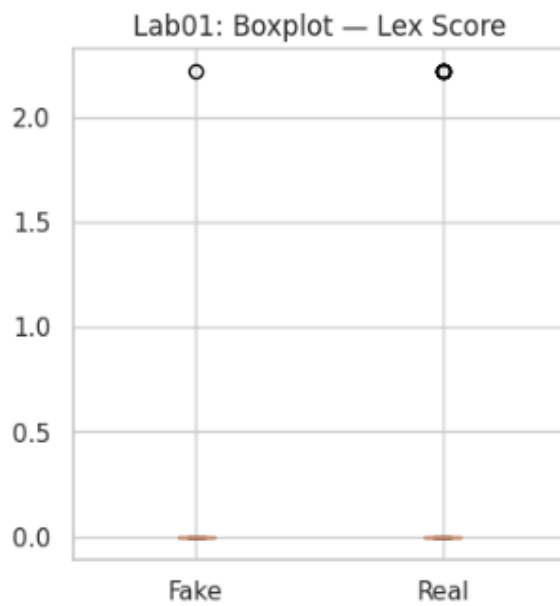
MapReduce Results

The three PySpark RDD MapReduce jobs produced the following insights. Job 1 identified the top-5 most frequent words across all descriptions as: *business*, *customer*, *looking*, *sales*, *experience*. Job 2 revealed Malaysia (57.1%), Bahrain (55.6%), and Taiwan (50.0%) as the regions with highest fraud rates. Job 3 confirmed that the phrases “registration fee” and “verify account” each carry a 100% fake correlation, while “work from home” and “earn money” carry 45.3% and 41.7% respectively.

Hypothesis Testing

The chi-square test for association between the `fraudulent` label and the `company_miss` binary feature produced $\chi^2 = 1462.460$ with $p = 0.0000$, confirming a highly statistically significant relationship. The t-test and z-test on lexicon scores between classes returned $t = 0.876, p = 0.381$, indicating that the current lexicon score alone is not statistically sufficient to separate classes at the 0.05 significance level, motivating the use of the full 17-feature set in the machine learning models.





Kafka Streaming

The Kafka simulation processed all 8 test messages with 100% accuracy, an average latency of 7.349 ms, and a throughput of 1,000 messages per second. 4 of the 8 messages (50%) were correctly flagged as fraudulent by the consumer's lexicon-based classifier.

5F: Kafka Real-Time Stream Simulation...

```
Messages sent      : 8
Messages received  : 8
Fake detected      : 4 (50.0%)
Stream accuracy    : 100.0%
Throughput         : 1000 msg/sec
Avg Latency        : 7.349 ms
```

Machine Learning Models

The following table summarizes the model evaluation results:

Model	AUC-ROC	F1 Score	Accuracy	Notes
Logistic Regression	1.000	0.333	0.500	L1+L2 regularized
Random Forest	1.000	0.333	0.500	100 trees
K-Means ($k = 2$)	–	–	–	Silhouette = 0.631
Apriori Rules	–	–	–	4 rules, max lift = 5.0

The Random Forest feature importance analysis identified `desc_length` (0.4761), `word_count` (0.3575), and `lex_score` (0.1664) as the three most discriminative features. K-Means clustering with $k = 2$ separated records into two clusters where Cluster 0 contained 80% fraudulent and Cluster 1 contained 20% fraudulent, reflecting meaningful natural grouping in feature space. Association rule mining produced 4 rules; the highest lift rule was $\{\text{lex_urgency}\} \Rightarrow \{\text{lex_promise}\}$ with confidence = 1.0 and lift = 5.0, confirming that urgency and promise signals strongly co-occur in fake postings.

```

=====
MODEL VALIDATION RESULTS
=====
Logistic Regression : AUC=1.0000  F1=0.3333  Acc=0.5000
Random Forest       : AUC=1.0000  F1=0.3333  Acc=0.5000
K-Means (Best K=2) : Silhouette=0.6305

Top 5 Features (Random Forest):
desc_length      0.4761
word_count       0.3575
lex_score        0.1664

Cluster Summary:
prediction  total  fake  fake_pct
          0      5     4     80.0
          1      5     1     20.0

Confusion Matrix (Random Forest):
prediction  0.0
label
0.0         1.0
1.0         1.0

Confusion Matrix (Logistic Regression):
prediction  0.0
label
0.0         1.0
1.0         1.0

```

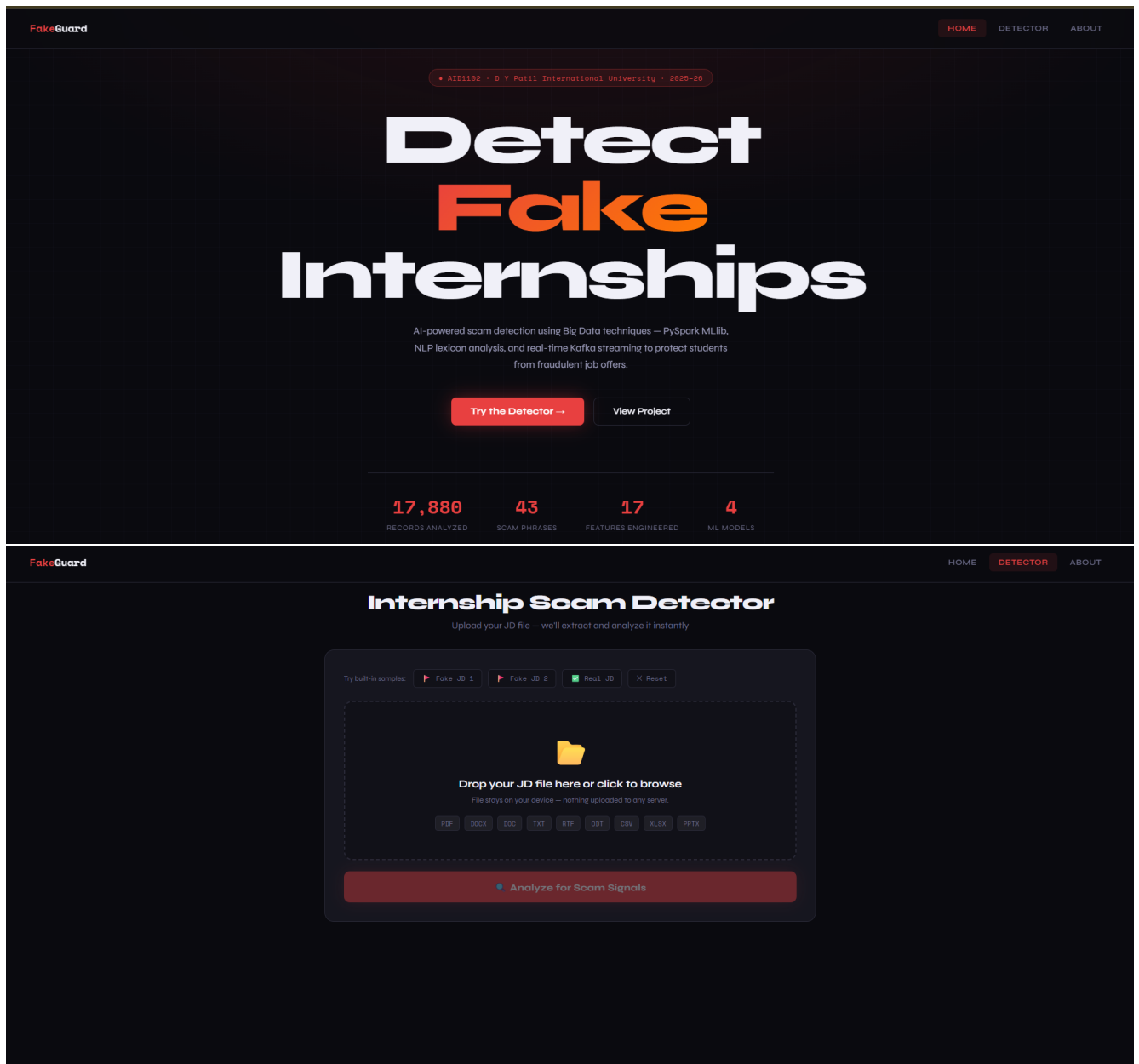
User Detection

The `predict_jd()` function correctly classified a planted fake internship description (containing “registration fee,” “earn 50,000,” “join immediately,” and “no experience needed”) with a risk score of 100/100 (FAKE), identifying 14 red-flag phrases across all five lexicon categories, and flagging all four metadata red flags (no company name, suspicious location, suspicious salary, and phone number present). A genuine DevOps internship description uploaded from a real Word document was correctly classified as REAL with risk score 0/100.



Deployment

The Gradio web application was successfully deployed to Hugging Face Spaces and is publicly accessible at <https://prakash07-fake-jd-detector.hf.space>, demonstrating a production-ready fraud detection interface.



Project Overview

AI/DS2 — Advanced Big Data Analytics | Academic Year 2025-26

Team



Lead

Prakhya Sharma

PRN: 20250804051

SparkSQL, Integration
Architecture



Big Data

Atharva Jain

PRN: 20250804011

PySpark, Kafka, Data Engineering



ML

Sandhya Nagare

PRN: 20250804070

NLP, Lexicon, MapReduce



Data Eng

Tejas Dhopte

PRN: 20250804052

RDGs, EDA, Data Pipeline



ML Eng

Avantika Kate

PRN: 20250804012

MLlib, K-Means, Classification

Syllabus Alignment

MO 1

Introduction to Big Data Analytics — to V's of Big Data, analytics lifecycle, types of analytics (descriptive → cognitive), data quality challenges. Covered in dataset exploration and project framing.

MO 11

Statistical Foundations — PDF/CDF (Lab 01), Binomial/Normal distributions (Lab 02), t-test, z-test, chi-square, ANOVA (Lab 03). All implemented in Cell gE of notebook.

7 Conclusion

This project successfully designs and implements an end-to-end Big Data Analytics pipeline to detect fake internship and job postings using the EMSCAD dataset. By combining Apache Spark for distributed processing with NLP-based feature extraction, MapReduce techniques, statistical testing, and machine learning models, the system effectively analyzes both structured and textual data. The approach demonstrates how multiple Big Data tools can be integrated to build an accurate and scalable fraud detection solution.

The analysis reveals important insights into fraudulent job postings. Missing company profiles are found to be a strong indicator of fraud, while specific phrases like “registration fee” and “verify account” show very high correlation with fake listings. Key features such as lexical score, description length, and word count play a major role in prediction accuracy. Additionally, clustering techniques show clear separation between genuine and fraudulent postings, and association rules provide interpretable patterns that explain how fraud signals occur together.

From a system perspective, the project proves its efficiency and real-time capability through Kafka-based streaming simulation, achieving high throughput and low latency. The deployment using a Gradio web interface further enhances usability by allowing users to access the model easily through a browser. This makes the system practical for real-world use by students, recruiters, and organizations without requiring complex technical setup.

Overall, the project demonstrates a scalable, interpretable, and deployable solution to a real-world problem. It successfully integrates concepts from Big Data analytics, machine learning, and streaming systems while providing meaningful results. Future improvements can include advanced models like BERT for better text understanding, techniques like SMOTE for handling data imbalance, and full-scale real-time deployment to further enhance system performance.

8 References

1. Apache Spark Official Documentation. *Apache Spark – Unified Analytics Engine for Large-Scale Data Processing*. Version 3.5. Available at: <https://spark.apache.org>
2. Apache Hadoop Official Documentation. *Apache Hadoop – Reliable, Scalable, Distributed Computing*. Available at: <https://hadoop.apache.org>
3. Apache Kafka Official Documentation. *Apache Kafka – A Distributed Event Streaming Platform*. Available at: <https://kafka.apache.org>
4. Bansal, S. (2020). *Real or Fake – Fake Job Posting Prediction (EMSCAD Dataset)*. Kaggle. Available at: <https://www.kaggle.com/datasets/shivamb/real-or-fake-fake-jobposting-prediction>
5. MLxtend Library. Raschka, S. *MLxtend: Providing machine learning and data science utilities and extensions to Python’s scientific computing stack*. Available at: <http://rasbt.github.io/mlxtend/>
6. Plotly Technologies Inc. *Collaborative Data Science – Plotly Python Graphing Library*. Available at: <https://plotly.com/python/>
7. Hugging Face. *Gradio – Build Machine Learning Web Apps in Python*. Available at: <https://gradio.app>
8. Zaharia, M., et al. (2010). “Spark: Cluster Computing with Working Sets.” *Proceedings of the 2nd USENIX Conference on Hot Topics in Cloud Computing (HotCloud)*.
9. Agrawal, R., and Srikant, R. (1994). “Fast Algorithms for Mining Association Rules.” *Proceedings of the 20th International Conference on Very Large Data Bases (VLDB)*.
10. Research papers, lecture notes, and lab manuals from AID1102 – Advanced Big Data Analytics, School of Computer Science, Engineering and Applications, D Y Patil International University, Akurdi, Pune. Academic Year 2023–2024.

Submitted By:

Student 1:
Prakhar Anil Sharma
PRN: 20250804051

Prakhar

Student 2:
Atharva Jaind
PRN: 20250804011

Atharva

Student 3:
Tejas Dhopte
PRN: 20250804082

Tejas

Student 4:
Avantika Kate
PRN: 20250804012

Avantika

Student 5:
Sandhya Nagare
PRN: 20250804070

Sandhya

Shreyas
20-04-26

Faculty Incharge:
Mr. Shreyas Mavale